

智慧水发·蜀水云采 ERP — 项目技术手册

本手册由「技术栈详解」与「新手入门」合并而成，是本仓库**唯一的技术文档**：前半是架构参考（每个机制讲透，带文件锚点），后半是操作手册（按学习路径组织）。配套阅读根目录 `CLAUDE.md`（项目宪法，权威约定，覆盖任何默认行为）。

所有结论基于对仓库真实代码的逐文件探查（`package.json` ×9、`main.ts`、`portal-cookie.ts`、`envelope-crypto.ts`、`jwt-secret.helper.ts`、`schema.prisma` (90 model / 2134 行)、`ai-bid-analysis` 模块树、共享包源码），不是推测。所有 shell 命令在 `water-erp/` 目录下执行。

目录

第一部分 · 总览与概念

- 1. [一句话定位](#)
- 2. [核心概念入门](#)（5 个骨架概念 + 13 项工具科普）
- 3. [工程底座（Monorepo 与工具链）](#)

第二部分 · 技术栈深度（架构参考）

- 4. [后端 API — 系统心脏](#)（模块全景 / 启动序列 / 鉴权 / 数据层 / 信封加密 / 实时异步 / `ai-bid-analysis` / LLM / 安全测试）
- 5. [前端门户矩阵](#)
- 6. [共享包](#)
- 7. [基础设施](#)
- 8. [AI 与外部服务](#)
- 9. [端到端请求流](#)
- 10. [投标阶段状态机](#)
- 11. [技术债清单](#)

第三部分 · 上手手册（按学习路径）

- 路径 A: [把它跑起来](#)（A0–A8）
- 路径 B: [看懂一个门户](#)（B1–B6）
- 路径 C: [改后端](#)（C1–C10，含 NestJS 三件套、依赖注入基础、请求生命周期、新增接口实战、Prisma 迁移流程）

第四部分 · 深度专题

- 专题 1: [NestJS 依赖注入深挖](#)

- [专题 2: Prisma 迁移流程深挖](#)

附录

- [A. 命令速查](#)
- [B. 版本快照](#)
- [C. 下一步学习路线](#)

第一部分 · 总览与概念

1. 一句话定位

这是一个 **pnpm 单仓多门户** 的政企招投标+采购 ERP: **9 个前端门户** (7 个 Next.js 16 + React 19, 1 个 Vue 3 异类) + **1 个 NestJS 11 后端** (43 个模块 / 90 张表) + **1 个 Python OCR 微服务** + **3 个共享包** + **3 个 Docker 容器**, 靠「按门户命名的 httpOnly cookie」做多角色会话隔离, 靠 BullMQ + Redis 把昂贵的 LLM 投标分析甩到独立 worker 进程。

它不是"一个应用", 而是一个由统一 API 串联的**门户矩阵**。

2. 核心概念入门

这一节写给不熟悉本项目或某项技术的读者, 分两部分: 先建立 5 个**项目骨架概念**, 再用"是什么 + 本项目里用它做什么"的方式快速过 13 项**工具概念**。看完再回头看第二部分的深度细节就不怵了。

2.1 先建立 5 个项目骨架概念

1. **这是 monorepo**: 一个仓库里塞了 9 个前端 + 1 个后端 + 共享包, 统一管理 (见 § 3)。
2. **pnpm workspace**: 子项目之间用 `"@water-erp/shared": "workspace:*" 互相引用, 不用发布到 npm`。配置见 `water-erp/pnpm-workspace.yaml` (`apps/*` + `packages/*`, 排除 `apps/web-erp-old`)。
3. **门户 (portal)**: 每个角色一个独立前端站点 (信息门户、商城、供应商、采购管理台、专家、开标端…), 端口不同 (见 § 5)。
4. **cookie 隔离**: 每个门户一张独立登录 cookie (`token_web` / `token_supplier` …), 同一个浏览器里互不串。
5. **后端只有 1 个**: 所有门户都调同一个 NestJS API (:4001)。

2.2 工具概念入门（13 项）

pnpm — 包管理器

和 `npm`、`yarn` 同类，职责是**下载、安装、版本锁定项目依赖的第三方库**。三个杀手锏：① 全局 store + 硬链接（大库只存一份，省磁盘、装得快）；② 严格的 `node_modules` 结构（杜绝“幽灵依赖”——没用却 import 得到的库）；③ **原生支持 monorepo**。

本项目选它的主因就是第 ③ 点：`water-erp/` 是 pnpm workspace，9 个 app + 3 个共享包靠 `"@water-erp/shared": "workspace:*"` 互相引用，`pnpm -r build` 递归构建、`pnpm --filter api test` 精准在单包跑测试。`package.json` 里 `"packageManager": "pnpm@10.33.0"` 锁死版本，保证全团队用同一个 pnpm。

Node.js + TypeScript — 运行时 + 类型语言

Node.js 让 JavaScript 能脱离浏览器、直接在服务器上跑（整个 API 后端就是它驱动的）。**TypeScript** 是 JavaScript 的超集，加了**静态类型**——写代码时就能发现“传错参数”“拼写错字段”这类错误，而不是等运行时崩。

本项目：API（NestJS）和所有 Next.js/Vue 门户都是 TypeScript。`tsconfig.json` 关了 `esModuleInterop`（只开 `allowSyntheticDefaultImports`），所以对 `sanitize-html` 这类老式导出的库要用 `import x = require('pkg')`——这是一个真实的坑。

NestJS — 后端框架

一个有“架构主见”的 Node.js 后端框架，核心是**模块（Module）+ 依赖注入（DI）+ 装饰器**。把功能拆成一个个模块（`BidModule`、`AuthModule` …），需要什么服务就声明注入，框架自动装配。相比“一个文件一把梭”的 Express，它更像 Java 的 Spring，强约定、好维护。

本项目：`apps/api/src` 下有 **43 个模块**。鉴权用全局 `AuthGuard` → `RolesGuard` 链（通过 `@Roles('bid_expert')` 这样的装饰器声明权限），`@Public()` 标记免登录接口。`main.ts` 里统一装全局管道、过滤器、helmet、CORS。

Prisma — ORM（数据库访问层）

ORM（对象关系映射）让你用 **TypeScript 代码操作数据库，而不是手写 SQL**。在 `schema.prisma` 里声明数据模型（`model User { id String ... }`），Prisma 自动生成带类型的查询客户端，写 `prisma.user.findFirst({where:{...}})` 就能查库，字段错了编译期就报错。

本项目：`schema.prisma` 有 **90 个 model、2134 行**，是整个系统的数据底座。改模型后跑 `pnpm db:migrate` 生成迁移、`pnpm db:generate` 重新生成客户端。种子（`pnpm db:seed`）是**破坏性**的——会先 `TRUNCATE` 清空再重载。

PostgreSQL — 关系型数据库

主流的开源关系型数据库，数据存成一张张**有结构的表**（像 Excel），表之间用外键关联。适合需要**强一致性、复杂关联查询、事务**的业务——招投标系统的钱、合同、评分这种不能错的数据正合适。

本项目：跑在 Docker 容器 `postgres:16-alpine`，端口 5432。所有业务数据（项目、供应商、投标、评分、公告…）都在它里面。

Redis — 内存数据库（缓存 + 队列后端）

一个极快的**内存型**键值存储，数据放内存里所以读写都在毫秒级以下。常用于两件事：① 缓存（把慢查询的结果存起来重复用）；② **任务队列的后端**。

本项目：Redis 7 跑在容器里（端口映射 `6380→6379`，刻意非默认端口）。它身兼两职——既做缓存，又是 BullMQ 任务队列的存储后端。API 和独立 worker 都读 `REDIS_URL`。

BullMQ — 任务队列

解决“**有个活又慢又重，不想让用户干等**”的问题。比如 LLM 分析一份标书要好几十秒、花真金白银——把任务“丢进队列”立刻返回，后台的 worker 进程慢慢消费。队列本身存在 Redis 里。

本项目：per-item AI 投标分析用 BullMQ。关键设计是 worker 跑在**独立进程**（`ai-bid-analysis-worker.module.ts`，刻意不被主 API 引入），这样又慢又重的 LLM 调用不会拖垮 API。改了代码后必须**手动 kill 重启 worker**，`pnpm dev` 的热重启管不到它。

MinIO — 对象存储

专门存**文件**（PDF、图片、Word…）的服务，API 兼容亚马逊 S3。为什么不存数据库？因为数据库存大文件又慢又臃肿；对象存储为海量文件而生，按“桶（bucket）+ key”存取。

本项目：跑在容器里（9000 服务 / 9001 控制台）。投标文件、上传附件都进 MinIO，`FileAsset` 表只存元数据（文件名、大小、key）。投标文件还做了**信封加密**（见 § 4.5）——DB 泄露也不等于文件可解密。

Next.js + React — 前端框架

React 是写 UI 的库：把界面拆成可复用的组件，数据变了界面自动更新。**Next.js** 是 React 之上的框架，补齐了路由、服务端渲染、打包、代码分割等工程能力——让你专注写页面而不是配脚手架。

本项目：9 个门户里有 **7 个用 Next.js 16 + React 19**（信息门户、商城、采购管理台、专家门户、开标端、水叮当、大屏）。Next 16 默认用 Turbopack 打包（比 webpack 快）。唯一的异类是供应商门户用 Vue，见下。

Vue 3 + Element Plus — 另一套前端体系（供应商门户）

Vue 是和 React 并列的另一套写 UI 的框架（语法风格不同，有人偏爱它的模板写法）。**Element Plus** 是 Vue 生态里成熟的组件库（表格、表单、弹窗现成可用）。

本项目：`apps/supplier-portal`（:3004）是**唯一的 Vue 门户**（Vue 3.5 + Vite 8 + Element Plus + Pinia）。它不消费团队 React 共享组件包 `@water-erp/ui`，是复用体系上的一个断层点。

Tailwind CSS — 原子化 CSS 框架

不写传统 CSS 类，而是用一堆**现成的工具类**直接在 HTML 上拼样式：`className="rounded-2xl p-4 text-sm flex"`。好处是不用起类名、样式即所见、没有死 CSS。v4 用新的 `lightningcss` 引擎，编译更

快。

本项目：所有 Next.js 门户都用 **Tailwind v4**。 `.impeccable.md` 定义了团队设计系统（海军蓝→冰蓝调色板、1px 发丝线、新拟态凸起边框），共享 React 组件在 `packages/ui`。

Docker + docker-compose — 容器化

Docker 把一个软件和它的运行环境打包成"容器"，换台机器也能一模一样地跑，告别"在我电脑上能跑"。
docker-compose 用一个 `docker-compose.yml` 文件**一键起多个容器**并连好它们。

本项目： `docker-compose.yml` 一把起三个基础设施—— `pnpm infra:up` 同时启动 PostgreSQL、Redis、MinIO， `pnpm infra:down` 停掉， `pnpm infra:logs` 看日志。前端/后端代码本身没容器化，开发时直接 `pnpm dev` 跑。

JWT + Cookie — 登录鉴权

JWT（JSON Web Token） 是登录后服务器发给客户端的一张"电子凭证"，里面编码了"你是谁、什么角色"，每次请求带上它服务器就能认人，不用每次查库。**Cookie** 是浏览器自动随请求带上的小块数据，把 JWT 放进 httpOnly cookie 既安全（JS 读不到）又自动携带。

本项目最特别的地方：**每个门户一张独立 cookie**

（ `token_web` / `token_supplier` / `token_expert` / `token_mall` ），靠 `X-Portal` 请求头 + Origin 端口推断该读哪张——这样同一个浏览器里多个门户各自独立登录、互不串。生产环境 JWT 密钥缺失或太短会**直接拒绝启动**。

3. 工程底座（Monorepo 与工具链）

维度	选型	实证
包管理器	pnpm 10.33.0 （ <code>packageManager</code> 字段锁定）	<code>water-erp/package.json:5</code>
内部包引用	<code>workspace:*</code> 协议	见各 app <code>dependencies</code>
脚本编排	concurrently 9.2 ，8 进程并行 + 彩色标签	<code>water-erp/package.json:7</code>
构建	<code>pnpm -r build</code> （原生递归， 无 Turborepo/Nx ）	<code>water-erp/package.json:18</code>
语言	TypeScript（根 <code>^6</code> ，app <code>^5</code> / <code>^5.7</code> ）；seed 走 tsx	<code>apps/api/package.json:82</code>
后端 tsconfig	<code>"module": "commonjs" + "target": "ES2021" + <code>allowSyntheticDefaultImports</code>（无 esModuleInterop）</code>	<code>apps/api/tsconfig.json:3,8,9</code>

⚠ **无 esModuleInterop 的 import 约定**：对 CJS 函数导出包（`module.exports = fn`，如 `sanitize-html`、`helmet`）必须用 `import x = require('pkg')` ——默认 import 会编译成 `.default` → 运行时 `undefined`。这是真实踩过的坑（`main.ts:5` 用 `import helmet from 'helmet'`，靠 `allowSyntheticDefaultImports` 编译期放行）。

⚠ **大屏 (bigscreen) 游离于体系外**：不在 `pnpm dev` 内，需 `pnpm dev:bigscreen`，端口 3010 硬编码，不在 `packages/config/ports.ts` 单一真相源里。

第二部分 · 技术栈深度（架构参考）

4. 后端 API（`apps/api` :4001）— 系统心脏

4.1 模块全景

`apps/api/src` 下 43 个模块目录，分四类：

- **业务域**：`bid`（招投标全生命周期，核心）、`expert`、`supplier` / `supplier-portal`、`announcement`、`catalog`（商城目录）、`budget`、`procurements` / `procurement`（新/旧并存）、`project-management`、`progress`、`work-arrangements`、`contacts`
- **AI/文档**：`ai`、`ai-bid-analysis`（per-item LLM，独立 worker）、`local-ai`（全局 LLM/OCR/Embedding 网关）、`assistant`（水叮当）、`tender-write` / `tender-review` / `tender-sample` / `tender-history`、`knowledge`（RAG 语料）、`search`
- **平台设施**：`auth`、`prisma`、`redis`、`storage`（MinIO）、`upload`、`notification`（多通道投递）、`audit` + `operation-log`（双层审计）、`scheduler`（cron）、`verification`（短信/电话核验）、`alerts`、`user-settings`、`imports`、`dashboard`
- **公共**：`common`（过滤器/守卫/装饰器/加密/消毒/jwt-helper）

4.2 启动序列（逐行解读 `main.ts`）

`apps/api/src/main.ts`（98 行）做了 6 件事，每件都有工程理由：

1. **进程级兜底**（`main.ts:12-18`）：注册 `uncaughtException` / `unhandledRejection`，记录崩溃原因但不 `process.exit` ——交给 Nest 自愈，避免静默退出。
2. **trust proxy**（`main.ts:39-41`）：读 `TRUST_PROXY`（默认 `loopback`）。反代后必须设 `=1`（信任一跳），否则 `req.ip` 全是代理 IP → 限流/审计失真。

3. **全局管线** (`main.ts:43-46`): `setGlobalPrefix('api')` + `cookieParser()` + `ValidationPipe({whitelist, transform})` + `HttpExceptionFilter`。
4. **helmet CSP** (`main.ts:51-71`): 手工配置每条指令——`scriptSrc` / `styleSrc` 放 `unsafe-inline` (Swagger UI + Tailwind v4 内联样式)、`imgSrc` 含 `data:` / `blob:` / `https:` (MinIO + data URI)、`connectSrc` 含 `wss:` / `ws:` (WebSocket 开标)、`objectSrc:'none'`、`upgradeInsecureRequests` (HSTS 配套)。`crossOriginEmbedderPolicy:false` 刻意关掉避免破坏跨域资源。
5. **CORS** (`main.ts:20-31, 73-77`): `corsOrigins()` 优先读 `CORS_ORIGINS` (逗号分隔真实域名), 缺失时从 **PORTS** 派生所有本地门户端口 (含 `127.0.0.1`) ——端口重分配自动同步, 无需手改。
6. **Swagger 门控** (`main.ts:80-89`): 仅 `NODE_ENV !== 'production'` 挂 `/api/docs` ——生产隐藏避免接口面泄露。

4.3 鉴权——三层模型 + 三级 cookie 解析 (全栈最精妙处)

这不是"一个 JWT 中间件", 而是**多门户会话隔离系统**。

角色 → 门户 → cookie 命名

`auth/portal-cookie.ts:11-18` 的 `ROLE_PORTAL` 把 6 种角色归到 4 个 cookie 命名空间:

```
admin / bid_host / procurement_staff → web → token_web
supplier                               → supplier → token_supplier
bid_expert                             → expert → token_expert
mall                                   → mall → token_mall
```

关键设计: `bid-portal(:3007)` 没有 `token_bid` —— `admin` / `bid_host` 复用 `token_web` 命名空间 (`portal-cookie.ts:31` 把 3007 显式映射到 `'web'`), 使无 `X-Portal` 头的客户端请求 (如 AppShell 的 `/auth/me`) 也能读到 `token_web`。代价是 `bid-portal` 的登录要绕经 `expert-portal(:3006)` 登录页设 cookie。

三级 token 解析 (`tokenFromRequest`, `portal-cookie.ts:71-78`)

- 1) `X-Portal` 请求头 ← 显式声明, 最可信
- 2) `Origin/Referer` 端口推断 ← 由 `PORTS` 派生的 `PORT_TO_PORTAL` 反查
- 3) `legacy`token` cookie` ← 兜底 (Swagger 直连场景)

同名账号靠门户区分 (`PORTAL_ROLE_PRIORITY` , `auth/auth.service.ts:15-20`)

```
mall:      ['mall']
supplier:  ['supplier']
web:       ['procurement_staff','leader','staff','bid_host','admin']
expert:    ['bid_expert','bid_host','admin']
public:    [几乎所有角色]           // 公共门户入口，放行所有
```

配合 schema 的 `@unique([username, role])` 复合唯一约束：三个 role 不同的账号可共用登录名「陈主任」/ `czr@2026`，登录时按 `X-Portal` 来源门户在对应优先级列表里匹配——电子商城→mall、采购管理端→procurement_staff、开标端→bid_host。

鉴权执行链

```
请求 → AuthGuard(全局, APP_GUARD) → 解析 tokenFromRequest → jwt.verify
      → @Public() ? 放行
      → RolesGuard(全局) → 查 @Roles(...) 元数据 → 无元数据=任意已登录用户可访问
      → [业务层] 自定义 Guard(Procurement/Owner/Admin) + service ownership
```

审计盲区：`AuthGuard` / `RolesGuard` 在 `OperationLog` 拦截器之前执行（守卫先于拦截器），故 401/403 拒绝不会被记录到操作日志——只有失败登录会记。

JWT 密钥生产守卫 (`common/jwt-secret.helper.ts`)

`NODE_ENV=production` 时 `JWT_SECRET` 缺失或 <32 字符 → 抛 **FATAL 拒绝启动** (`:19-25`)。开发态回退到固定弱值 `'water-erp-jwt-secret'`（不随机生成——随机值会让重启后已签发 token 全失效，开发体验差），并打醒目告警。

4.4 数据层——Prisma 6 + PostgreSQL 16, 90 张表

`apps/api/prisma/schema.prisma` (2134 行 / 90 model) 的领域建模以 **BidProject** 为 hub (`schema.prisma:250-287`)，向外辐射 16 条关系：suppliers / openingSession / openingRecords / experts / scoreItems / clarifications / supervisionLogs / archiveItems / aiAnalysisTask / evaluationResults / reviews / scoreReviews / invalidBids / supplierSubmissions / procurementProject ...

几个值得细看的子图：

- **评分子图** (当前分支 `feat/expert-invalid-bid` 正在改)： `BidScoreItem` → `BidScorePoint` → `BidScoreRecord` → `BidScorePointDecision` (专家逐项打点) + `BidScoreReview` (核对关口) + `BidScoreDelta` (变更) + `BidInvalidBid` (无效投标)。最近 5 个 commit 都围绕"核对关口 verify-score gate (`REVIEW_PENDING`)"和"无效投标实时流转"——这是活跃开发区。

- **AI 分析子图**: `AiBidAnalysisTask` → `AiBidderResult` / `AiConcordanceResult` / `AiBidReport` , per-item 粒度, 由独立 worker 写入。
- **供应商全生命周期**: `Supplier` → `Contact` / `Qualification` / `Classification` / `Evaluation` / `ChangeRecord` / `BidSubmission` / `CatalogApplication` 。
- **冗余快查字段**: `BidSupplier.bidValidity` (:300 , "valid"|"invalid" , 由 `BidInvalidBid` 驱动) —— 为避免每次列表查询都 join 无效投标表而刻意冗余。

种子是**幂等且破坏性**的: `TRUNCATE ... RESTART IDENTITY CASCADE` 全清后从 `prisma/seed-data/*.json` 快照重载 (专家库 186 名来自真实库导出)。

4.5 投标文件信封加密 (`common/crypto/envelope-crypto.ts`)

这是投标保密性的核心, **不是死代码**。解决的问题: AES-256-GCM 的 DEK (数据加密密钥) 原本以 hex 明文存 DB → DB 泄露 = 所有投标文件可解密。

方案 (KMS 信封加密):

1. `deriveKEK()` (:20-25): `SHA-256(固定salt + KMS_SECRET)` 派生 KEK (密钥加密密钥)。
`KMS_SECRET` 仅在本文件被读取, **永不写 DB**。
2. `wrapKey(dek, kmsSecret)` (:33-44): 用 KEK 走 AES-256-GCM 加密原始 DEK, 输出 `iv(12B) + authTag(16B) + ciphertext` 的 base64 blob 存库。
3. `unwrapKey(blob, kmsSecret)` (:52-75): 解包恢复 "hexkey:hexiv:hextag" 格式的 DEK, 再用 DEK 解密文件。
4. `isWrappedKey()` (:81-86): 通过正则区分原始 hex DEK vs 已包裹的 base64 blob, **支持平滑迁移** (老数据未包裹、新数据已包裹, 按需补裹)。

`KMS_SECRET` 缺失 → 运行时抛 `KMS_SECRET is not configured`。 `supplier-portal.service.ts` 在投递/解密时封口/拆封。

4.6 实时与异步——Socket.IO + BullMQ 双通道

- **实时 (Socket.IO 4.8)**: `@nestjs/websockets` + `@nestjs/platform-socket.io` , 开标大厅推送供应商解密状态 + 倒计时。前端 bid-portal / expert-portal 用 `socket.io-client` 。
- **异步 (BullMQ 5.79)**: per-item AI 投标分析任务队列。

BullMQ worker 为何是独立进程

`apps/api/src/ai-bid-analysis-worker.module.ts` 是一个**独立的 Nest bootstrap** (`AiBidAnalysisWorkerModule`), **刻意不被 AppModule 引入**。它只装配 worker 需要的东西:

```
ConfigModule + BullModule(Redis) + PrismaModule + RedisModule
+ LocalAiModule + StorageModule + AiBidAnalysisModule + AiBidQueueModule
+ providers: [TenderProcessor, BidderProcessor]
```

工程理由：

- **资源隔离**：LLM 调用慢且重（单任务 \$ 级、分钟级），不能阻塞 API 的事件循环。
- **独立扩缩容**：可只加 worker 副本不加 API 副本。
- **复用领域代码**：与 API 共享 `AiBidAnalysisModule` 的全部 service，不重复实现。

⚠ **运维陷阱**：`pnpm dev` 的 API `--watch` 重启**不会**重启独立 worker。改 `ai-bid-analysis` 源码后必须**手动 kill + 重跑** `pnpm --filter api dev:worker:ai-bid-analysis`。且重跑分析会创建**新 job**——别依赖稳定 `jobId`。

4.7 ai-bid-analysis 内部——LLM 输出硬化（亮点）

这个模块（`apps/api/src/ai-bid-analysis/`）规模惊人，且对 LLM 输出做了**多层验证**（不是裸调 LLM 拿啥用啥）：

- **8 个 prompt 模板**（`prompts/`）：`tender-requirements` / `bidder-info` / `requirement-matching` / `item-scoring` / `comparative-scoring` / `competitive-analysis` / `clarification-draft+summary` / `score-criteria` / `price`。
- **16 个 service**（`services/`）：`tender-extractor` / `bidder-extractor` / `requirement-matcher` / `generic-item-scorer` / `comparative-scoring` / `competitive-analysis` / **`concordance-verifier`**（一致性核对） / **`excerpt-verify`**（原文摘录核验） / **`fraud-detector`**（围标串标检测） / `price-analyzer` / `score-criteria-inferer` / `docx-generator`（生成 Word 报告） / `report-generator` / `document-metadata-extractor` / `plaintext-fetcher` / `system-data-aggregator`。
- **utils/ 校准层**：`calibration.ts`（评分校准） / `neutralize.ts`（中立化去偏见） / `qualification.ts` / `text-similarity.ts` / `price-statistics.ts`。
- **两个 processor**：`tender.processor.ts`（标书侧） + `bidder.processor.ts`（投标人侧），均带 `.spec.ts`。

这层“验证器 + 校准器”是区别于“裸调 LLM”demo 的关键——`concordance-verifier` + `excerpt-verify` 强制 LLM 结论必须能映射回原文摘录，降低幻觉。

4.8 LLM 调用未收口（已知技术债）

`local-ai/LlmService` 是集中的 DeepSeek/vLLM 网关（还含 `OcrService`、`EmbeddingService`（RAG）、`LlmOutputValidator`、`VllmMonitorService`）。但多个模块**绕过它直接**
`fetch(process.env.DEEPSEEK_API_URL)`：

- `announcement-ai` / `ai/supplier-selection-ai` / `ai/ai.service` / `expert/expert-extraction-ai` / `assistant/deepseek.provider`

后果：改 `LlmService`（如换模型、加超时/重试/审计）**不影响这些直连模块**。这是需要偿还的债。

4.9 安全与测试清单

维度	实现
密码哈希	bcryptjs (cost 10)
密码重置	PasswordChangeRequest / PasswordResetRequest 两张表 (token 流转)
XSS 消毒	sanitize-html 写时消毒公告 content , DTO @Transform
限流	@nestjs/throttler : login 10/min、register 5/min
安全头	helmet (见 4.2)
加密	信封加密 (见 4.5) + 国密 sm-crypto (投标加解密)
审计	AuditLog (操作) + OperationLog (全局拦截器, 每次 API 请求) 双层
单测	Jest 30 + ts-jest, co-located *.spec.ts (ai-bid-analysis 覆盖最密)
E2E	Jest + supertest, cookie auth + 种子数据, --forceExit (Nest 残留 Redis/BullMQ handle)

⚠️ **jest + ESM-only 依赖 (pnpm 环境):** jest.config.js + test/jest-e2e.json 用**两条互补的 transformIgnorePatterns** (pnpm 的 .pnpm/<pkg>@<ver>/node_modules/<pkg>/ 路径含两段 node_modules , 单正则则会回溯误判) 转译 ESM-only 依赖 (htmlparser2 / @paralleldrive/cuid2 / @noble/hashes)。新增 **ESM-only 依赖报 Cannot use import statement** → **加进两份 allowlist**。

5. 前端门户矩阵

5.1 双栈现状 (精确版本)

门户	:port	框架	角色	关键依赖差异
public-portal	3002	Next 16.2.3 / React 19.2.4	公开	基线
mall	3003	同上	mall	+ framer-motion 12
web	3005	同上	procurement_staff	最重: 见下
expert-portal	3006	同上	bid_expert	+ socket.io-client / react-resizable-panels
bid-portal	3007	同上	admin/bid_host	+ socket.io-client

门户	:port	框架	角色	关键依赖差异
assistant	3008	同上	公开	全屏聊天 + 视频背景
bigscreen	3010	同上	—	独立启动，游离体系外
supplier-portal	3004	Vue 3.5 + Vite 8	supplier	唯一异类

5.2 Next.js 派统一基线

- **Next.js 16.2.3 App Router + Turbopack**（默认，已移除 `--webpack`；`lightningcss` 1.32 + Node 24.16 已解 Turbopack CSS 回归）
- **React 19.2.4** / React DOM 19.2.4
- **Tailwind CSS v4**（`@tailwindcss/postcss`）
- **lucide-react**（图标，1.5px 描边）+ **sonner**（toast）
- 消费 `@water-erp/config` / `shared` / `ui`

5.3 web（采购管理工作台）——功能最复杂的门户

额外引入：

- **@tanstack/react-query 5**（服务端状态缓存）
- **react-hook-form 7 + zod 4 + @hookform/resolvers**（表单 + schema 校验）
- **echarts 6 + recharts 3**（双图表库——echarts 给复杂可视化、recharts 给常规图）
- **radix-ui + shadcn 4.6**（无头组件 + 组件生成器）
- **framer-motion / motion 12**（动画）
- **react-markdown + remark-gfm**（AI 输出 Markdown 渲染）
- **exceljs**（导出 Excel）
- **Playwright**（E2E，仅此门户有）

5.4 supplier-portal —— 唯一 Vue 门户

刻意选型（`apps/supplier-portal/package.json`）：

- **Vue 3.5 + Vite 8**（`@vitejs/plugin-vue`）
- **Element Plus 2.14** + `@element-plus/icons-vue`
- **Pinia 3**（状态）+ **Vue Router 5**
- **axios + dayjs**

它不消费 `@water-erp/ui`（React 包），是断层点；但共享 `@water-erp/config`。

5.5 设计系统 (`.impeccable.md`)

工业精度美学：

- 1px 发丝分割线、等宽数字 (monospace numerals)
- 海军蓝→冰蓝分层调色板 (页面 BG `oklch(0.975 0.012 258)`)
- 新拟态凸起边框系统：定向明暗阴影对 (由 BG 派生)，按钮 hover 凸起、active 内陷——反模式：禁渐变按钮、禁 emoji 当图标、禁 Material 全向 elevation
- `rounded-2xl` 卡片 / `rounded-xl` 按钮

6. 共享包 (`packages/`)

6.1 `@water-erp/config`

- `ports.ts` : `PORTS` 对象 (8 项, api 4001 + 7 门户) ——全仓库端口唯一真相源, `main.ts` 的 CORS、`portal-cookie.ts` 的端口反查都从它派生。
- `urls.ts` : `portalURL(app, path)` (含 SSR 安全的 `window` 检测)、`ROLE_PORTAL` (角色→门户)、`ROLE_LANDING` (角色→登录后路径)、`landingURL(role)` 。

6.2 `@water-erp/shared`

`index.ts` 导出 5 个域：`types` (领域类型) / `constants` (状态标签/映射/品牌常量) / `workbench` (工作台约定) / `bid-events` (Socket 事件契约) / `design-tokens.css` (设计令牌)。编译到 `dist/`，改后必须 `build`。

6.3 `@water-erp/ui`

6 个 React 工作台组件：`MetricCard` / `PageHero` / `SectionCard` / `StatusBadge` / `DataToolBar` + `utils.ts` (含 `cn` helper)。Tailwind v4 消费方需在 `globals.css` 加 `@source` `"../../node_modules/@water-erp/ui"`。

7. 基础设施 (`docker-compose.yml`)

服务	镜像	宿主端口	备注
PostgreSQL	<code>postgres:16-alpine</code>	5432	带健康检查 + 持久卷
Redis	<code>redis:7-alpine</code>	6380 →6379	AOF 持久化；刻意非默认端口，API + worker 都读 <code>REDIS_URL</code> (缺省回退 <code>localhost:6380</code>)

服务	镜像	宿主端口	备注
MinIO	<code>minio/minio:latest</code>	9000 + 9001 (console)	S3 兼容，存投标文件/上传

8. AI 与外部服务

8.1 OCR 微服务（`services/ocr` :8100）—— Python 独立栈

与 JS 栈完全解耦的 **FastAPI + uvicorn** 服务：

- **FastAPI 0.115 + uvicorn[standard] 0.34**
- **rapidocr_onnxruntime**（OCR，rapid 系而非 PaddleOCR）+ **PyMuPDF / pdf2image / Pillow / numpy**（PDF/图像）
- **opendaler-pdf[hybrid]**（PDF 加载）
- `start.sh` 首次运行自动建 `.venv`；`pnpm dev:ocr` 启动
- API 经 `local-ai/OcrService` 消费

8.2 LLM

- **DeepSeek**（`DEEPSEEK_API_URL` / `DEEPSEEK_API_KEY` / `DEEPSEEK_MODEL=deepseek-v4-flash`）：供电叮当助手 + 投标分析。
- **vLLM** 备选（`local-ai/VllmMonitorService` 监控）。
- **EmbeddingService**（RAG 向量）+ **LlmOutputValidator**（结构化输出校验）。

9. 端到端请求流（把以上串起来）

以"专家在专家门户提交评分"为例（从浏览器到 DB 的**宏观**链路；NestJS **内部**的逐步生命周期见 [§ 路径C-C3](#)）：

① 浏览器 (expert-portal :3006)

↓ fetch 带 cookie token_expert + 头 X-Portal: expert

② Nest API :4001 /api/...

↓ AuthGuard(全局)

③ tokenFromRequest(req):

X-Portal=expert → 读 cookie token_expert → jwt.verify

↓ RolesGuard(全局) 查 @Roles('bid_expert')

④ 业务 Guard (Owner/Admin) + service ownership 校验

↓ [OperationLog 拦截器此时才介入—守卫已过, 故 401/403 不被记录]

⑤ Controller → Service → Prisma(90 表) → PostgreSQL

↓ (评分写 BidScoreRecord; 若触发无效投标, 写 BidInvalidBid + 更新 BidSupplier.bidValidity 冗余字)

⑥ HttpExceptionHandler 统一错误格式 / 成功响应

↓ (可选) BullMQ 入队 ai-bid-analysis → 独立 worker 进程消费

⑦ 响应回浏览器

10. 投标阶段状态机

DOWNLOAD → SUBMIT → OPENING → EVALUATING → ARCHIVED

- 同态转移幂等
- 非法转移抛 `ConflictException` (409)
- 评分核对关口 (`REVIEW_PENDING`) 是 `EVALUATING → ARCHIVED` 间的 gate (当前活跃开发区)

11. 技术债与改进清单 (附实证)

#	债	实证 / 影响
1	LLM 调用未收口	5+ 模块绕过 <code>LlmService</code> 直连 DeepSeek, 换模型/加审计失效
2	双图表库并存	web 同时装 echarts 6 + recharts 3, bundle 膨胀
3	三处旧代码	<code>water_erp_web/</code> (静态原型)、 <code>apps/web-erp-old/</code> (旧 Next.js)、 <code>AGENTS.md</code> (端口文档过时)
4	大屏游离	bigscreen 端口 3010 硬编码, 不在 <code>PORTS</code> 真相源

#	债	实证 / 影响
5	Vue 门户断层	supplier-portal 不消费 @water-erp/ui ，组件无法复用
6	新旧采购模块并存	Procurements （新，有 Prisma） vs Procurement （旧，无 Prisma）
7	审计盲区	守卫先于拦截器 → 401/403 不进 OperationLog

第三部分 · 上手手册（按学习路径）

三条路径按需取用，不用从头读到尾。想先建立概念再看本部分：回 [§ 2 核心概念入门](#)。

我想…	直接跳到	你会学到
把它在本机跑起来	路径 A	装依赖 → 起基础设施 → 配置 → 迁移 → 启动 → 登录验证 + 踩坑排查
看懂某个前端门户	路径 B	Next.js App Router + 目录结构 + 前端怎么调后端 + 登录态保持 + 一次完整请求追踪
改后端 / 加接口	路径 C	NestJS 三件套 + 依赖注入基础 + 请求生命周期 + 新增接口实战 + Prisma 迁移流程

路径 A：我想把它跑起来

A0. 环境准备

必备	版本	验证命令
Node.js	24.x （实测 v24.16.0）	<code>node -v</code>
pnpm	10.33.0 （被 packageManager 锁定）	<code>pnpm -v</code>
Docker + Docker Compose	任意现代版	<code>docker -v</code>

没装 pnpm? `npm install -g pnpm@10.33.0` (或用 corepack: `corepack enable && corepack prepare pnpm@10.33.0 --activate`)。

A1. 拿到代码、装依赖

```
cd water-erp
pnpm install
```

`pnpm install` 会：

- 读所有子项目的 `package.json`，下载第三方依赖到全局 store + 硬链接进各 `node_modules/`
- 按 `pnpm-workspace.yaml` 把内部包 (`@water-erp/*`) 连成软链接
- 执行 `onlyBuiltDependencies` 里列的原生模块构建 (prisma、esbuild、sharp…)

A2. 起基础设施 (PostgreSQL / Redis / MinIO)

```
pnpm infra:up
```

等价于 `docker compose up -d`，一把起三个容器：

容器	端口	用途
<code>water-erp-postgres</code>	5432	主数据库
<code>water-erp-redis</code>	6380 →6379	缓存 + BullMQ 队列后端
<code>water-erp-minio</code>	9000 / 9001 (控制台)	文件存储

验证：`docker ps` 看三个容器都在 Up；`pnpm infra:logs` 看日志；`pnpm infra:down` 停。

A3. 配置 `.env` (唯一一个 `.env` 文件)

编辑 `apps/api/.env`，它只有 10 个键 (实测)：

```
DATABASE_URL=postgresql://water_erp:water_erp_dev@localhost:5432/water_erp
JWT_SECRET=随便一串至少32字符的密钥          # 生产 <32 字符会拒绝启动
KMS_SECRET=随便一串主密钥                      # 投标文件信封加密用；空则解密投标时报错
REDIS_URL=redis://localhost:6380

# AI（不配则 AI 功能不可用，但不影响其他）
DEEPSEEK_API_URL=https://api.deepseek.com
DEEPSEEK_API_KEY=sk-xxxx                      # 水叮当 + 投标分析必需
DEEPSEEK_MODEL=deepseek-v4-flash

# 可选（有缺省值）
DEEPSEEK_BASE_URL=                            # 仅 local-ai/LlmService 读
OCR_SERVICE_URL=http://localhost:8100
SMS_DEBUG_BYPASS=true                          # 短信验证码自动通过为 "123456"
```

哪些是**硬必需**：`DATABASE_URL`（连库）、`JWT_SECRET`（生产 ≥ 32 字符）、`KMS_SECRET`（涉及投标文件加解密时运行时报错）。哪些**有缺省**：`REDIS_URL`（缺省 `localhost:6380`）、`OCR_SERVICE_URL`、`SMS_DEBUG_BYPASS`。哪些**只为 AI**：`DEEPSEEK_*`。

A4. 编译共享包（最容易漏的一步）

```
pnpm --filter @water-erp/shared build
pnpm --filter @water-erp/config build
```

`packages/shared` 和 `packages/config` 编译产物在 `dist/`，**改过它们必须重新 build**，否则其他项目引用的是旧产物。漏了这步会报"找不到 `@water-erp/shared` 导出"。

A5. 数据库迁移 + 种子

```
pnpm db:migrate      # 执行 Prisma 迁移（当前共 58 个）
pnpm db:seed         # 灌种子数据
```

⚠️ **db:seed 是破坏性的**：它会先 `TRUNCATE ... RESTART IDENTITY CASCADE` 清空所有业务表，再从 `apps/api/prisma/seed-data/*.json` 重载。**别在生产库上跑**。

种子账号（密码统一 `<用户名>@2026`，详见 CLAUDE.md 「Seed Data」表）：

账号	密码	角色	去哪个门户
陈主任	czr@2026	procurement_staff	采购管理工作台 :3005
陈主任	czr@2026	bid_host	开评标管理端 :3007
陈主任	czr@2026	mall	采购商城 :3003
supplier1	supplier1@2026	supplier	供应商门户 :3004
刘苡池（任一专家名）	expert@2026	bid_expert	专家门户 :3006

三个「陈主任」共用登录名、靠 X-Portal 头区分角色——这是 `@unique([username, role])` 复合唯一约束带来的设计。

A6. 启动应用

```
pnpm dev
```

它会用 `concurrently` 并行起 8 个进程，每个一种颜色标签：

```
[mall] :3003    [supplier] :3004    [web] :3005    [expert] :3006
[public] :3002  [bid] :3007    [api] :4001    [assistant] :3008
```

注意： `pnpm dev` 不包含大屏，也不包含 AI 分析 worker 和 OCR。需要的话单独起：

```
pnpm dev:bigscreen                # 大屏 :3010
pnpm --filter api dev:worker:ai-bid-analysis # AI 投标分析 worker（独立进程，必需否则分析任务不出队）
pnpm dev:ocr                      # Python OCR 微服务 :8100
```

⚠ **worker 陷阱**： `pnpm dev` 的 API 热重启管不到独立 worker 进程。改了 `ai-bid-analysis/` 代码后，必须手动 **kill worker 再重启**，否则跑的是旧代码。

A7. 登录验证

- 浏览器打开 信息门户 `http://localhost:3002`
- 点登录，输入种子账号（如 陈主任 / `czr@2026`）
- 系统按角色自动跳转到对应门户（采购员→ `:3005/dashboard`、开标主持人→ `:3007/bid` …）

路由由 `packages/config/src/urls.ts` 的 `ROLE_PORTAL` + `ROLE_LANDING` 决定。

A8. 常见踩坑（Troubleshooting）

症状	原因	解决
<code>Cannot find module '@water-erp/shared'</code> 或导出错	共享包没编译	跑 A4 的两条 build
AI 投标分析一直 PENDING	worker 没起	单独起 <code>dev:worker:ai-bid-analysis</code>
改了 ai-bid 代码没生效	worker 是独立进程，热重启管不到	kill + 重启 worker
启动报 <code>lightningcss</code> native binding 错	Node 24 + 旧版 <code>lightningcss</code>	确认 <code>lightningcss ≥ 1.32.0</code> （本项目已修）
<code>KMS_SECRET is not configured</code>	没配 <code>KMS_SECRET</code>	在 <code>.env</code> 填任意值（开发）
生产启动 FATAL: JWT_SECRET	密钥缺失或 <32 字符	配 ≥32 字符的 <code>JWT_SECRET</code>
单测报 <code>Cannot use import statement</code>	新增了 ESM-only 依赖	加进 <code>jest.config.js</code> + <code>test/jest-e2e.json</code> 两份 allowlist
跨端口登录态丢失	cookie 是按端口命名的	确认前端发了 <code>X-Portal</code> 头；下载链接别加 <code>rel=noreferrer</code>
端口被占用	之前进程没退干净	<code>lsof -i :4001</code> 找到 PID kill 掉

路径 B：我想看懂某个门户

以**采购管理工作台 `apps/web`（:3005）**为例（功能最全），其它 Next.js 门户结构一致。

B1. Next.js App Router 速成（够用版）

- Next.js 的页面放在 `src/app/` 目录，目录路径 = URL 路径：`src/app/login/page.tsx` → `/login`。
- `page.tsx` 是页面组件，`layout.tsx` 是包裹布局（侧边栏、顶栏常放这）。
- 路由组（route group）：`src/app/(main)/dashboard/` 里的 `(main)` 不进 URL，只用来分组共享同一套 `layout.tsx`。所以 `(main)/dashboard` 的 URL 是 `/dashboard`。
- `src/middleware.ts` 是请求级守卫，在页面渲染前跑（见 B4）。

B2. 门户目录结构

```
apps/web/
├─ next.config.ts      # 配置: /api 代理到后端、打包优化
├─ src/
│  └─ app/             # 路由 (App Router)
│     └─ login/        # 登录页
│        └─ (main)/    # 登录后受保护区域 (共享 layout)
│           └─ dashboard/    ← /dashboard 驾驶舱
│           └─ notice/      ← 信息发布
│           └─ supplier/    ← 供应商管理
│           └─ expert/      ← 专家管理
│           └─ mall-management/
│           └─ bid-analysis/ ← AI 投标分析
│           └─ tender-write/ · tender-review/ ← 招投标文档
│           └─ procurements/ · projects/ · progress/ · work-arrangements/
│              └─ ...
│  └─ components/      # 本门户私有组件
│  └─ lib/              # 工具: API 封装、常量
│     └─ api.ts         ← ★ 前端调后端的统一入口
│     └─ middleware.ts  # ★ 登录守卫
└─ package.json
```

B3. 前端如何调后端 API（本节最关键）

前端不直接访问 `localhost:4001`，而是走同源 `/api/...`，由 Next.js 转发到后端。两层机制：

第一层： `next.config.ts` 的 rewrite（代理）

```
// apps/web/next.config.ts
async rewrites() {
  return [{ source: "/api/:path*", destination: `${API_SERVER}/api/:path*` }];
}

// API_SERVER 默认 http://localhost:4001
```

浏览器请求 `/api/bid/projects` → Next.js 服务器转发到

`http://localhost:4001/api/bid/projects`。**好处：**前端和后端同源（都走 :3005），cookie 自动携带，无跨域问题。

第二层：src/lib/api.ts 的统一封装

```
// apps/web/src/lib/api.ts (节选)
const BASE = '/api';
const PORTAL = 'web';

async function fetchApi<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`${BASE}${path}`, {
    credentials: 'include', // ★ 带上 cookie
    ...init,
    headers: { 'X-Portal': PORTAL, ...init?.headers }, // ★ 告诉后端"我来自 web 门户"
  });
  if (!res.ok) {
    // 解析后端统一错误格式 { statusCode, code, error, ... }
    throw new ApiError(res.status, body.code, body.error);
  }
  return res.json();
}

export const api = {
  get: <T>(p: string) => fetchApi<T>(p),
  post: <T>(p: string, body) => fetchApi<T>(p, { method: 'POST', headers: { 'Content-Type': 'appli
  // put / patch / delete / postForm 同理
};
```

两个细节决定一切：

- `credentials: 'include'` → 浏览器自动带上 `token_web` cookie。
- `'X-Portal': 'web'` → 后端 `portal-cookie.ts` 据此知道该读 `token_web`、按 web 门户的角色优先级鉴权。

完整调用链（从点击到数据）

```
页面组件 api.get('/bid/dashboard-stats')
  ↓ fetch('/api/bid/dashboard-stats', {credentials, X-Portal:web})
Next.js (:3005) rewrite 转发
  ↓ http://localhost:4001/api/bid/dashboard-stats (cookie token_web 随行)
NestJS AuthGuard 解析 X-Portal → 读 token_web → jwt.verify
  ↓ RolesGuard 查 @Roles
BidController.getDashboardStats()
  ↓
BidService.getDashboardStats() → prisma.bidProject.findMany(...)
  ↓
PostgreSQL
  ↓ JSON 原路返回
页面渲染
```

B4. 登录态怎么保持

两层保险：

1. **src/middleware.ts**（请求级守卫）：每个匹配的请求前先跑，检查 `token_web` cookie 是否存在，没有就 302 跳 `/login?redirect=原路径`。

```
// apps/web/src/middleware.ts
const AUTH_COOKIE_NAME = 'token_web';
if (!token) return NextResponse.redirect(loginUrl); // 带 redirect 参数
```

`matcher` 排除 `/login`、`/api/*`、静态资源、根路径。

2. **页面级检查**：每个页面内部还会自己校验一次（middleware 是兜底，防 cookie 在客户端导航中丢失）。

B5. 走一遍真实页面

以驾驶舱为例（`src/app/(main)/dashboard/page.tsx`）：

1. 用户访问 `/dashboard`
2. `middleware.ts` 检查 `token_web` 存在 → 放行
3. `(main)/layout.tsx` 渲染侧边栏 + 顶栏
4. `dashboard/page.tsx` 里调 `api.get('/bid/dashboard-stats')` 拉统计

5. 经 B3 的链路拿到 JSON，用 `MetricCard`（来自 `@water-erp/ui`）渲染卡片

想看哪个模块，就在 `src/app/(main)/` 下找对应目录；想知道它调了哪些接口，grep 该目录里的 `api.get/post/`。

B6. Vue 门户（supplier）有什么不同

`apps/supplier-portal`（:3004）是唯一的 Vue 门户，差别：

维度	Next.js 门户（如 web）	supplier-portal
框架	Next.js + React	Vue 3 + Vite
UI 库	Tailwind + <code>@water-erp/ui</code>	Element Plus
状态	React hooks / context	Pinia
HTTP	<code>src/lib/api.ts</code> （fetch）	axios
路由	App Router（文件即路由）	Vue Router（配置式）
cookie	<code>token_web</code>	<code>token_supplier</code>
X-Portal	<code>web</code>	<code>supplier</code>

但它同样调 `:4001` 后端，只是 cookie 换成 `token_supplier`、`X-Portal` 换成 `supplier`。鉴权链路完全一致。

路径 C：我想改后端

C1. NestJS 三件套：Module / Controller / Service

NestJS 的核心抽象是三个角色，看真实的 `bid` 模块：

① Module (`bid.module.ts`) — 装配清单

```
@Module({
  imports: [                                // 依赖别的模块
    AuthModule, PrismaModule, NotificationModule,
    BullModule.registerQueue({ name: QUEUE_NAMES.TENDER_PROCESSING }),
    AiBidAnalysisModule,
  ],
  controllers: [BidController],             // 暴露的 HTTP 路由
  providers: [BidService, BidGateway, ClarificationAiService, ScorePointExtractorService], //
  exports: [BidGateway, BidService, ClarificationAiService], // 允许别的模块用我的服务
})
export class BidModule {}
```

Module 就是一张"这个功能块有哪些控制器、哪些服务、依赖谁、对外暴露谁"的**清单**。

② Controller (`bid.controller.ts`) — 路由层

```
@ApiTags('开评标管理')
@ApiCookieAuth('token')
@Controller('bid')                        // 前缀 /bid
@Roles('admin', 'bid_host', 'procurement_staff', 'leader', 'staff') // 整个控制器的角色限制
export class BidController {
  constructor(
    private readonly bidService: BidService,           // 构造器注入服务
    private readonly scorePointExtractor: ScorePointExtractorService,
  ) {}

  @Get('projects')                               // GET /api/bid/projects
  @ApiOperation({ summary: '项目列表' })
  listProjects(@Query('stage') stage?: string | string[]) {
    return this.bidService.listProjects(stages);       // 委托给 service
  }
}
```

Controller 只管接 **HTTP 请求**、**转交 service**，不写业务逻辑。

③ Service (`bid.service.ts`) —— 业务逻辑层

```
@Injectable()
export class BidService {
  constructor(
    private prisma: PrismaService,           // 注入数据库访问
    private notificationService: NotificationService,
    @Optional() private readonly clarificationAi?: ClarificationAiService, // 可选注入
    @Optional() private readonly gateway?: BidGateway,
    @InjectQueue(...) private readonly tenderQueue: Queue,
  ) {}
  // 方法里写真正的业务逻辑, 用 this.prisma.xxx 操作数据库
}
```

`@Injectable()` 标记"我可以被注入"。Service 写业务、调 Prisma、发通知、入队列。

口诀：Module 是清单，Controller 接请求，Service 干活。

C2. 依赖注入 (DI) 基础 ★

这里讲**够用的基础用法**。想彻底搞懂 (token 本质、四种提供方式、作用域生命周期) 看 [专题 1](#)。

什么是依赖注入？为什么需要它？

不用 DI 时，你要用一个类得自己 `new`：

```
const prisma = new PrismaService();    // 自己造
const svc = new BidService(prisma);    // 自己把依赖塞进去
```

问题：依赖一多，`new` 套 `new`，每处都要管"先造谁、传给谁"，改一个构造器签名全项目跟着改。

DI 的做法：你只在构造器上**声明**"我需要 PrismaService"，NestJS 自动造好塞给你：

```
constructor(private prisma: PrismaService) {}    // 只声明，不 new
```

NestJS 内部有个 **DI 容器** (IocContainer)，启动时扫描所有 `@Module()`，把 `providers` 列表里的类实例化成**单例**，谁构造器声明要就自动注入。

DI 的三个关键机制（本项目实证）

机制一：providers + exports 决定"可见性"

Module A 想注入 Module B 的 service，**必须**把 B 放进 `imports`，且 B 的 service 在 B 的 `exports` 里。看 `bid.module.ts`：它 `imports: [NotificationModule]`，而 `NotificationModule` 内部 `exports: [NotificationService]`，所以 `BidService` 能注入 `NotificationService`。

机制二：@Global() 模块免导入

```
// apps/api/src/prisma/prisma.module.ts
@Global()                                // ★ 全局模块
@Module({ providers: [PrismaService], exports: [PrismaService] })
export class PrismaModule {}
```

`@Global()` 让 `PrismaService` **全局可见**——任何 service 都能直接 `constructor(private prisma: PrismaService)`，**不用在每个模块的 imports 里重复写 PrismaModule**。这就是为什么 `BidService` 没显式 import `PrismaModule` 却能用 `PrismaService`。同理 `LocalAiModule`、`StorageModule` 也是全局的。

机制三：全局守卫用 APP_GUARD 注册

```
// app.module.ts 的 providers
providers: [
  { provide: APP_GUARD, useClass: AuthGuard },      // 全局鉴权
  { provide: APP_GUARD, useClass: RolesGuard },    // 全局角色
  { provide: APP_GUARD, useClass: ThrottlerGuard }, // 全局限流
],
```

`APP_GUARD` 是个特殊 token，注册的守卫对**所有路由**生效，不用每个控制器手动加。

@Optional() 注入

```
@Optional() private readonly clarificationAi?: ClarificationAiService,
```

表示"有就注入，没有也不报错"。用于可选依赖（比如 AI 服务挂了，核心功能仍能跑）。

为什么 JwtModule.register 不是死代码？

一个真实陷阱：`auth / audit / user-settings` 三处都单独 `JwtModule.register({...})`，看着冗余。但**全局 AuthGuard 在被守卫模块的作用域内解析 JwtService**——删了这些 register，启动时 DI 容器找不到 `JwtService` 直接崩溃。**别当重复代码删。**

C3. 一个请求的完整生命周期

这是 NestJS **内部**的逐步生命周期（与 [§ 9 端到端请求流](#) 的宏观视角互补）。

```
HTTP 请求进来
↓
① 中间件 (cookie-parser、helmet)
↓
② 守卫 Guards (顺序: AuthGuard → RolesGuard → ThrottlerGuard)
  - AuthGuard: 解析 X-Portal → 读对应 cookie → jwt.verify
  - @Public() 直接放行; @Roles 校验角色; 不满足 → 401/403
↓
③ 拦截器 Interceptors 前半 (如 OperationLogInterceptor 记录开始)
↓
④ 管道 Pipes (全局 ValidationPipe: whitelist + transform, 按 DTO 校验参数)
↓
⑤ Controller 方法 (@Get/@Post + 参数装饰器 @Body/@Param/@Query/@CurrentUser)
↓
⑥ Service 业务逻辑 → Prisma → PostgreSQL
↓
⑦ 拦截器后半 (OperationLog 记录成功 + 耗时)
↓
⑧ 异常过滤器 (HttpExceptionFilter: 统一错误为 {statusCode, code, error, timestamp, path})
↓
响应
```

⚠ **审计盲区**：守卫（②）在拦截器（③）**之前**执行，所以 **401/403 拒绝不会被 OperationLog 记录**（失败登录会记，因为走的是另一条路径）。审计别只 grep `@Roles`。

C4. 动手实战：新增一个业务接口

假设要加"获取供应商收藏列表"接口 `GET /api/supplier/favorites`：

- 数据模型**：确认 `schema.prisma` 里有没有 `UserFavorite` model（本项目已有）。没有就加（见 C5）。
- 写 DTO**（如需入参校验）：`apps/api/src/supplier/dto/xxx.dto.ts`，用 `class-validator` 装饰器。
- 写 service 方法**：在 `SupplierService` 里加 `getFavorites(userId: string)`，用 `this.prisma.userFavorite.findMany(...)`。

4. 写 controller 方法:

```
@Get('favorites')
@Roles('procurement_staff', 'admin')
getFavorites(@CurrentUser() user: { userId: string }) {
  return this.supplierService.getFavorites(user.userId);
}
```

5. 注册: 新模块的话加进 `app.module.ts` 的 `imports` (已有模块省略这步)。

6. 写测试: `supplier.service.spec.ts` 加用例。

7. 前端调用: `apps/web/src/lib/api/...` 里加 `api.get('/supplier/favorites')`。

C5. Prisma 迁移流程详解 ★

这里讲操作流程。想搞懂迁移内部机制(影子库、`_prisma_migrations` 表、lock 文件)看 [专题 2](#)。

数据模型的真相在 `schema.prisma`

`apps/api/prisma/schema.prisma` (90 model / 2134 行) 是**唯一真相源**。改库结构 = 先改这里。

标准三步: 改 schema → 生成迁移 → 应用

```
# 1. 编辑 schema.prisma, 加/改 model
# 2. 生成并应用迁移 (开发环境)
pnpm db:migrate      # = prisma migrate dev
# 3. 重新生成 Prisma Client (让 TS 类型同步)
pnpm db:generate
```

`prisma migrate dev` 会:

- 对比 `schema.prisma` 和数据库差异
- 生成一个新的 `prisma/migrations/<时间戳>_<名字>/migration.sql`
- 在数据库执行这段 SQL
- 更新 `_prisma_migrations` 追踪表

迁移文件长什么样？（本项目真实例子）

```
-- apps/api/prisma/migrations/20260624141402_ai_bid_per_item_refactor/migration.sql
CREATE TYPE "AiAnalysisTaskStatus" AS ENUM ('PENDING', 'TENDER_PROCESSING', ...);
ALTER TYPE "AiBidderStatus" ADD VALUE 'CONCORDANCE_CHECKING';
ALTER TABLE "BidScoreItem" ADD COLUMN "criteriaSource" TEXT, ADD COLUMN ...;
ALTER TABLE "ai_bid_analysis_tasks" DROP COLUMN "createdBy", ADD COLUMN ...;
```

就是**纯 SQL**，由 Prisma 根据 schema 差异自动生成。本项目已累积 **58 个迁移**（在 `prisma/migrations/` 目录），按时间戳顺序应用。

⚠️ 非交互环境的坑（CI / 本工具环境）

`prisma migrate dev` 是**交互式的**——它会停下来问你"新迁移叫什么名字"。在非交互环境（CI、本工具）会卡住或报错。官方绕法（CLAUDE.md 已记录）：

```
# 方案 A：只生成不应用，再手动应用，最后标记
pnpm --filter api exec prisma migrate dev --create-only    # 只生成 SQL 不执行
pnpm --filter api exec prisma db execute --file <migration.sql>    # 手动执行
pnpm --filter api exec prisma migrate resolve --applied <迁移名>    # 标记为已应用

# 方案 B：直接给"危险操作授权"环境变量
PRISMA_USER_CONSENT_FOR_DANGEROUS_AI_ACTION=1 pnpm db:migrate
```

迁移 vs `db push`

	<code>prisma migrate dev</code>	<code>prisma db push</code>
生成 SQL 文件	✅ 有迁移记录	❌ 直接改库不留记录
适用	正式开发 / 生产	原型快速迭代
本项目用	✅ （58 个迁移）	—

本项目只用 `migrate`，因为招投标数据要可追溯、可回滚、可在生产重放。

schema 漂移（drift）怎么办

如果有人手动改了数据库（绕过迁移），`schema.prisma` 和真实库不一致 = 漂移。本项目有个迁移叫 `20260622153001_reconcile_drift` ——就是专门用来对账的。遇到漂移：用 `prisma migrate diff` 生成差异 SQL，再 `resolve`。

C6. 鉴权与权限：怎么保护你的接口

四个装饰器（来自 `common/decorators/` 和 `auth/`）：

装饰器	作用	例子
<code>@Public()</code>	免登录（跳过 AuthGuard）	登录、注册、公开公告
<code>@Roles('admin','bid_host')</code>	限制角色（RolesGuard 校验）	开标管理端接口
<code>@CurrentUser()</code>	拿当前登录用户	<code>@CurrentUser() user: {userId, role}</code>
不加任何	任意已登录用户都能访问	通用查询

记住：cookie 解析靠 `X-Portal` 头 → `portal-cookie.ts` 三级解析（头 → Origin 端口 → legacy token）。

C7. 实时通信（Socket.IO 网关）

```
// apps/api/src/bid/bid.gateway.ts
@WebSocketGateway({ cors: true })
export class BidGateway { ... }
```

`@WebSocketGateway()` 把一个类标记为 WebSocket 端点。开标大厅用它推送供应商解密状态、倒计时。前端 bid-portal / expert-portal 用 `socket.io-client` 连接。

C8. 错误处理约定

别自己拼错误 JSON。直接 throw NestJS 内置异常，全局 `HttpExceptionFilter` 会归一化为 `{ statusCode, code, error, timestamp, path }`：

```
throw new BadRequestException({ error: '参数错误', code: 'INVALID_PARAM' });
throw new ConflictException({ error: '阶段不能跳转', code: 'INVALID_STAGE' }); // 状态机非法转移 →
throw new NotFoundException({ error: '项目不存在' });
```

状态机非法转移（`DOWNLOAD` → `SUBMIT` → `OPENING` → `EVALUATING` → `ARCHIVED`）一律 throw `ConflictException`（409），同态转移幂等。

C9. 测试

类型	位置	命令
单元测试	co-located *.spec.ts (如 bid.service.spec.ts)	pnpm --filter api test
覆盖率	—	pnpm --filter api test:cov
E2E	apps/api/test/ (cookie auth + 种子数据)	pnpm --filter api test:e2e

⚠ **jest + ESM-only 依赖坑**: pnpm 的依赖路径含两段 node_modules，单正则 transformIgnorePatterns 会回溯误判，所以 jest.config.js 和 test/jest-e2e.json 用**两条互补正则**转译 ESM-only 依赖。新增依赖报 Cannot use import statement → 加进这两份 allowlist。E2E 带 --forceExit (Nest 残留 Redis/BullMQ handle 不退出)。

C10. 异步任务 (BullMQ)

重活 (LLM 分析) 甩给队列:

- 1. **入队**: service 里 this.tenderQueue.add('analyze', payload)
 - 2. **消费**: TenderProcessor / BidderProcessor (带 @Processor() 装饰器)
 - 3. **运行**: 跑在**独立 worker 进程** (ai-bid-analysis-worker.module.ts), 不被 AppModule 引入
- ```
pnpm --filter api dev:worker:ai-bid-analysis # 必需, 否则任务不出队
```

# 第四部分 · 深度专题

## 专题 1: NestJS 依赖注入深挖

基础用法见 § C2。本专题补充**底层机制**。

### Provider 的本质 = token + value

providers: [BidService] 是简写, 完整写法是:

```
providers: [
 { provide: BidService, useClass: BidService }, // token → 用哪个类实例化
]
```

- `provide` 是令牌 (token) (通常是类本身)
- `useClass` / `useValue` / `useFactory` 说明用什么值满足这个 token

## 四种提供方式

| 写法                                                          | 用途                    |
|-------------------------------------------------------------|-----------------------|
| <code>useClass</code>                                       | 默认，实例化一个类 (单例)        |
| <code>useValue: { foo: () =&gt; 1 }</code>                  | 直接给现成的值/mock 对象       |
| <code>useFactory: (dep) =&gt; ({...}, inject: [Dep])</code> | 需要运行时逻辑构造 (如读 env 配置) |
| <code>useExisting: OtherService</code>                      | 别名，让两个 token 指向同一实例   |

### `@Inject(token)` 什么时候要显式写?

构造器注入非类 token (字符串 token、Symbol) 时，TS 类型信息不够，必须显式：

```
constructor(@Inject('CONFIG') private config: ConfigType) {}
```

注入类 token 时可省略 ( `private prisma: PrismaService` 等价于 `@Inject(PrismaService)` )。

## 实例作用域：默认单例

NestJS provider 默认是单例——整个应用共享一个实例。可改为：

- `@Injectable({ scope: Scope.REQUEST })` → 每个请求新建 (性能差，谨慎用)
- `Scope.TRANSIENT` → 每次注入新建

本项目几乎所有 service 都是默认单例 ( `PrismaService` 全局单例，复用连接池)。

## DI 与守卫/拦截器/管道的关系

这些组件本身也是 provider，通过 `APP_GUARD` / `APP_INTERCEPTOR` / `APP_PIPE` 特殊 token 注册成全局。它们也能注入依赖 ( `AuthGuard` 注入 `JwtService`、`RolesGuard` 注入 `Reflector` 读元数据) ——这就是为什么删了 `JwtModule.register` 会启动崩溃。

## 一句话总结 DI

你声明依赖，容器负责造和塞。Module 控制"造什么、给谁看"，`@Global` 让全局可见，`APP_*` token 注册全局组件。

## 专题 2：Prisma 迁移流程深挖

操作流程见 [§ C5](#)。本专题补充内部机制。

### 迁移系统的组成

```
apps/api/prisma/
├─ schema.prisma # 唯一真相源 (model 定义)
├─ migrations/ # 迁移历史 (本项目 58 个)
│ └─ migration_lock.toml # ★ 迁移提供者锁 (防止 pg 和 mysql 混用)
│ └─ 20260610105504_init/
│ └─ migration.sql
│ ... 每个迁移一个目录
└─ seed-data/ # 种子 JSON 快照 (90 张表的种子)
```

### `migration_lock.toml` 是什么

```
provider = "postgresql"
```

锁定迁移提供者。Prisma 用它防止你在一个项目里混用 pg/mysql 迁移（会损坏历史）。

### `_prisma_migrations` 表

Prisma 在数据库里自动建一张 `_prisma_migrations` 表，记录"每个迁移是否已应用"（`migration_name` + `finished_at`）。`migrate dev` / `migrate deploy` 靠它判断哪些迁移要跑、哪些已跑。

### 影子数据库 (shadow database)

`migrate dev` 生成新迁移时，会临时建一个空库（影子库），在上面：

1. 重放所有已有迁移 → 得到"当前迁移应有的状态"
2. 对比 `schema.prisma` 差异 → 生成新迁移 SQL
3. 验证新迁移可重放



# 附录

## A. 命令速查

### # 安装与基础设施

|                              |                    |
|------------------------------|--------------------|
| <code>pnpm install</code>    | # 装依赖              |
| <code>pnpm infra:up</code>   | # 起 PG/Redis/MinIO |
| <code>pnpm infra:down</code> | # 停                |
| <code>pnpm infra:logs</code> | # 看日志              |

### # 共享包（首次或改后必做）

```
pnpm --filter @water-erp/shared build
pnpm --filter @water-erp/config build
```

### # 数据库

|                               |                          |
|-------------------------------|--------------------------|
| <code>pnpm db:migrate</code>  | # 执行迁移（58 个）             |
| <code>pnpm db:generate</code> | # 只重新生成 Prisma Client 类型 |
| <code>pnpm db:seed</code>     | # 灌种子（破坏性！）              |
| <code>pnpm db:studio</code>   | # Prisma Studio 可视化      |

### # 启动

|                                                           |                                 |
|-----------------------------------------------------------|---------------------------------|
| <code>pnpm dev</code>                                     | # 起 8 个核心门户 + API（不含 bigscreen） |
| <code>pnpm dev:bigscreen</code>                           | # 大屏 :3010（独立）                  |
| <code>pnpm --filter api dev:worker:ai-bid-analysis</code> | # LLM 分析独立 worker（必需）           |
| <code>pnpm dev:ocr</code>                                 | # Python OCR 微服务 :8100          |

### # 构建

|                                                              |          |
|--------------------------------------------------------------|----------|
| <code>pnpm build</code>                                      | # 全量     |
| <code>pnpm build:api   build:web   build:expert   ...</code> | # 单个 app |

### # 测试

|                                         |                          |
|-----------------------------------------|--------------------------|
| <code>pnpm --filter api test</code>     | # 单测                     |
| <code>pnpm --filter api test:cov</code> | # 覆盖率                    |
| <code>pnpm --filter api test:e2e</code> | # E2E (cookie auth + 种子) |

### # Lint

|                                     |                       |
|-------------------------------------|-----------------------|
| <code>pnpm lint</code>              | <code># 全工作区</code>   |
| <code>pnpm --filter api lint</code> | <code># 单个 app</code> |

## B. 版本快照（探查时点）

- pnpm **10.33.0** • Node **24** • TypeScript **5.7/6**
- NestJS **11** • Prisma **6.6** • PostgreSQL **16** • Redis **7** • MinIO latest
- Next.js **16.2.3** • React **19.2.4** • Tailwind **v4** • Vite **8** • Vue **3.5**
- Socket.IO **4.8** • BullMQ **5.79** • Jest **30**
- Python: FastAPI **0.115** • uvicorn **0.34** • rapidocr onnxruntime
- schema **90 model / 2134 行** • API **43 模块** • 前端 **9 门户** • 迁移 **58 个**

## C. 下一步学习路线

1. **跑通** → [路径 A](#) 完整走一遍，用种子账号登录每个门户点一遍。
2. **读懂** → [路径 B](#)，挑你最关心的模块（如开标 `bid`），从 `src/app/(main)/` 找页面，`grep api.` 看接口，回后端看 `controller`→`service`。
3. **能改** → [路径 C](#)，照 [C4](#) 实战加一个小接口，走完 `schema`→`service`→`controller`→测试→前端调用全链路。
4. **吃透原理** → [专题 1](#) + [专题 2](#)。
5. **查深度细节** → 回 [第二部分](#)（每节带文件锚点）。

提示：`CLAUDE.md` 是项目宪法——它明确写了哪些是“看着像死代码但不能删”（信封加密、`JwtModule.register`、`ts import` 约定），改之前先读它。